Vibe Coding — Session Notes

Source: *Day 2 | Generative AI Mastermind* — Team Outskill (YouTube: `DMh8_3D7SmI`)
Session: "Vibe coding — the messy reality of building with AI", presented by Cameron Sullivan
Covers: from the start of the vibe coding block (~3h45m) to the close of the session (~5h58m) — roughly 2 hours 12 minutes
Visuals: every distinct slide, whiteboard diagram and demo screen from this block is in the `vibe-coding-visuals/` folder (time-stamped filenames, `00-index-sheet.png` shows all of them at a glance)

The core argument

Vibe coding is **not** about being the most technical person in the room — it's about using the tool properly. Most people jump straight into a vibe coding tool, type "build me this", and burn time, money and credits building something nobody wants. The fix is a process: **explore, plan, then build.** Cameron's framing throughout: *"rule over AI, don't let AI rule over you."*

Today's build is deliberately ordinary — a **project management tool** — because it demonstrates every core piece of application building. The stated goal isn't to learn to build a project manager, it's to learn the structures and frameworks so you can build *any* application.

Who's in the room — three buckets: the entrepreneur (using AI to level up a business or sell solutions), the professional (upskilling in a current job), and the hustler (trying to leave bucket two for bucket one). All three need the same thing: organization.

What vibe coding is: the term comes from Andrej Karpathy — the iterative process of building full-fledged websites and applications without writing a line of code yourself. The most important word in that definition is **iterative**. It is not a one-click solution; you start small, make sure something works, then build on it. Expect mess, expect problems, troubleshoot them together.

---

1. Application architecture — the restaurant analogy

Before you build something, understand what you're building. You don't need to be a house-building expert, but you should know there's a foundation, plumbing and electrical. Four core pieces:

Piece	Analogy	What it means
Front end	The dining area / dining experience	What the customer interacts with — where you sit, eat and enjoy the restaurant
Back end	The kitchen	"Where the magic happens" — the food is prepared here
API	The waiter	How the order gets from the dining area to the kitchen and back. Also: a highway, the nervous system, the string between two tin-can phones. Formally, application programming interface — a way to get or give information between two places in your application or online
Database	The pantry	Where the ingredients are stored
Authentication	The reservation desk	Fine dining won't just let you in. The hostess checks her log — i.e. **the database** — confirms the name, time and party size, then lets you in. Think of it as the bouncer at a nightclub: it makes sure you are who you say you are, and that you're allowed in

The waiter (API) carries the order to the kitchen (back end); the chefs fetch ingredients from the pantry (database); the finished plate comes back out through the waiter to the dining area. People overcomplicate APIs — AI understands them well enough that you only need the basic mental model.

Why this matters for vibe coding: not so you become an engineer, but so that when something breaks you understand enough to tell AI what's wrong and steer it in the right direction.

---

2. Two ways to ideate an application

Before building, answer the real question: **is there demand?** Will enough people use it, and pay for it?

1. **Innovation** — take an existing application and make it better or make it your own. This is "taking inspiration", not stealing: you're not copying, you're seeing where the existing product falls short and making it unique. Big advantage: validated demand. Hundreds of project management tools already exist with huge customer bases and a public trail of complaints (too complicated, too expensive) — the market is proven.
2. **Invention** — the "unicorn": something brand new that's never been built before. A category of one, with more outsized gains relative to effort. Harder: nothing is battle-tested, and you don't know if a big enough market exists yet or will pay. (The Sam Altman "first one-person billionaire" idea lives here.)

For this build: route one — project management exists, we make it simpler and better.

---

3. The EPB framework: Explore → Plan → Build

The most important framework of the session. Most people skip straight to **Build** (marked red on the whiteboard as the bad path). That's how you waste time on things nobody needs, waste money, burn prompting credits, and spend twenty hours fixing something you never planned for — or worse, ship a full application nobody wants.

Thirty minutes in explore and plan can save you thirty hours in build.

The Pareto principle, applied to features

Eighty percent of users only use twenty percent of the features. So build the few things everyone needs, then iterate once you have feedback. The house analogy: AI makes it easy to get excited and throw up a house with a hot tub on the roof and a water slide around it — while the structural integrity is bad, the insulation is wrong, it's built on sand. Fanciness on top of a bad foundation is pointless.

Instagram started as a check-in app called Burbn. Nobody used the check-in feature; everybody loved the photo filters. They rebranded around the one thing that worked and grew from there into DMs, Stories, Reels. Most people try to build Instagram on day one instead of starting with the small core that people need.

---

4. Competitor research → a feature list (the Plan phase, live)

The hack: go to the **pricing pages** of the big competitors — Trello, Asana, monday.com. They do *value stacking*: every tier's features are listed side by side so buyers can compare. That makes it trivial to harvest a complete feature set — screenshot the pricing table, paste the screenshots into Claude.

Prompt used (paraphrased): *"I am a non-technical builder trying to build project management software via vibe coding. I've done research on my competitors and taken screenshots of their feature sets. Can you analyze all these screenshots and give me the complete set of features I need to build?"*

Why it's phrased that way: it sets context (non-technical → be my technical co-founder), states the goal, states the method (vibe coding), and states the input (screenshots of competitors).

Output — consolidated feature set across the three competitors: core task management (cards/tasks with assignees, due dates, priority, status), subtasks and checklists, multiple views (list, board/Kanban, calendar), boards and workspace hierarchy, collaboration, custom fields and columns, AI features, automations and workflows, integrations, reporting and dashboards, templates, time tracking, resource and portfolio management, and more.

Then the **boil-down prompt:** *"Since I'm a non-technical builder building my first application, give me a table of features in descending order of priority: two core features essential for any project management software, plus one differentiator feature where AI simplifies something very easily."*

The result — three features:
1. **Task and board management** — tasks with title, description, assignee, due date, status labels, dragged across a Kanban board. "The heartbeat of every project management tool — without it you don't have a product."
2. **Projects and list view** — group tasks into projects, view them as a sortable/filterable list or board.
3. **AI task generator (the differentiator)** — describe a project in plain language ("launch a podcast in six weeks") and AI creates the full set of tasks with suggested subtasks, due dates, dependencies and priorities.

Note: you don't need a Claude subscription — the same process works in ChatGPT or Gemini. Model choice is a preference; Cameron used Opus because newer models tend to overcomplicate brainstorming.

---

5. The PRD journey — three stages of talking to AI

A **PRD** is a product requirements document. Getting one is what separates a novice from someone experienced — the same process Cameron's team uses on six- and seven-figure client builds.

Stage 1 — the raw request. "Build me a project management app." This is where newbies live ("Newbieville"). You skip exploration and planning, and end up with something nobody wants.

Stage 2 — the basic PRD. The **Chinese restaurant analogy:** you order in English at a restaurant that speaks Mandarin. Your order comes back wrong — not because anyone is lazy, but because of the language barrier. Speak perfect Mandarin and you get exactly what you ordered. AI's language is **prompting**, and AI likes clear, detailed, structured prompts.

The PRD Claude produced — **TaskFlow AI MVP** — contained: product goal, target market (freelancers, solo founders, small teams of 2–5 who want to organize work without learning a complex tool), recommended tech stack, information architecture, data model, feature specs (projects, Kanban board view, task management, list view, AI task generator — "the primary differentiator and should feel magical"), and user flows.

Stage 3 — platform-optimized. Back to the restaurant: you both speak Mandarin but in different dialects. The order probably comes back right — but if you want the best chance of getting exactly what you asked for, speak the same dialect. Every AI tool has its own dialect, its own prompting style. So: take the PRD and rewrite it into a prompt for the specific tool.

How it was done: Google "Bolt.new best practices prompting", copy the whole official prompting guide page, paste it into Claude, and ask it to rewrite the PRD into a prompt that follows those guidelines. Cameron's point: **it's not about the tool.** Lovable, Replit, Base44, Emergent, Cursor, Claude Code — the principles carry over. Bolt.new was chosen because hosting, database and deployment live in one browser-based place.

---

6. Build phase — the live build in Bolt.new

Prompt 1 is the foundation. It sets up the architecture for the entire application — "build the foundation on concrete, not sand, or the first storm takes the house down." Prompt one covered stack, framework, folder structure and a placeholder homepage.

Two deliberate decisions in that first prompt:
- **Remove the database for now.** Bolt wanted to wire up Supabase immediately; Cameron stripped it out — local storage first, database later. One step at a time.
- **Design tokens up front** — clean minimal design, white background, Inter font, blue primary action colour.

He selected the **max model** (slower, more expensive, "thinks more intelligently") for architecture work.

Iteration is the job. The session includes real breakage — the presenter pasting the wrong clipboard contents into Bolt, prompts not landing, then a five-minute Pomodoro break while the build ran in the background. The lesson delivered in the moment: describe the issue to the AI and tell it what you need — *"when I click new project, nothing pops up… please make sure all CRUD works front to back with the database."* Switch to a faster model for small fixes.

---

7. Databases, illustrated with Spotify

Why do your saved songs survive signing out and back in? The database.

- Signing up creates you as a user and assigns a **user ID** — a universally unique identifier. Without it, thousands of people named Cameron Sullivan would have their libraries jumbled together. Same idea as a passport, driver's licence or school ID: a string unique to you.
- A database is **a table with rows and columns**, like an Excel or Google Sheet.
- These vibe-coded apps mostly use **relational databases**: multiple tables whose information is associated with one another. The Spotify illustration: *user_id → likes → song_id*, and separately *song_id → album, genre, artist*, and *album → release date, other songs on the album*. Each record has its own unique ID, so relationships are stored as IDs pointing at IDs.
- This is why you learn the basics: it's how you **test** your application.

CRUD — the four operations you test first:
- **C**reate — new information that didn't exist before
- **R**ead — viewing information that exists, on screen or in the database
- **U**pdate — editing existing information (like changing your username — not creating a new profile)
- **D**elete — "bon voyage, hasta la vista"

Before anything else, the task CRUD must work. Before the hot tub goes on the roof, the foundation has to be poured concrete. AI integration, team features, everything else comes after.

---

8. The build timeline

The plan of record shown on the deck slide, and followed live:

1. Setup (architecture prompt, project structure)
2. Database (link it in Bolt)
3. Basic user interface
4. **Test the task CRUD** — the gate before anything else
5. AI integration (task breakdown)
6. Team features
7. Authentication
8. Design pass, landing page, publish, Stripe

Linking the database: Bolt has a database button (stack-of-coins icon) → "ask Bolt to create a database". It uses a Bolt cloud database, which is Supabase underneath — scalable, secure, enterprise software runs on it — but managed inside Bolt.

Testing CRUD live: create a project ("Meeting Jeff Bezos"), read it, update it, delete it — checking each change in the projects table, where each record shows its unique ID, name, description, status, created-at and updated-at. Then the same for tasks (the tasks table also shows the **project ID** it belongs to). When drag-and-drop between columns wouldn't persist, the fix was a clear bug report to the AI: *"when I drag a task and drop it into a different column, it doesn't save and stay — please fix."*

Authentication (fourth core piece): the reservation desk, the hostess checking her log, the bouncer. In application terms: sign up, sign in, log out — make sure you are who you say you are and that you're allowed in. It checks the database every time.

Team features: add team members with role descriptions, then assign them to tasks. This closes the loop on the relational database lesson: in the tasks table, the task stores the **assignee ID** (e.g. `342d…`), which points back to the team member's record.

---

9. The AI feature — AI task breakdown

The feature: type a task in plain English ("launch a marketing campaign this week", "plan how to meet Jeff Bezos"), AI thinks and generates **four to ten sub-tasks** that you approve or deny, recommends the best team member for each, and adds the approved ones to the board automatically. The point: the last thing anyone needs from a project manager is more manual work.

Where AI models live — two places:
1. Inside the model maker's own app (ChatGPT) — where most people first used GPT models.
2. The **OpenAI API** — where you use those models inside *your own* application. The API is the highway: you send a prompt, you get a response.

The API key. How does OpenAI know you're allowed to use their models? An API key. It is:
- a **password** — never share it,
- an **E-ZPass / toll transponder** — a unique identifier that the scanner reads as you pass,
- and it's **connected to your money** — every call charges your account.

Getting the key: platform.openai.com → sign up, verify identity, add billing → API keys → create new secret key → name it → generate → copy and store it securely. Once you hit Done you can never see it again.

Storing it safely: not in email, not in Notion or Google Drive. A sticky note (nothing to hack) or a password manager. In code, there are two places: the `.env` **environment variables file**, or **secrets** — Bolt has secrets built in and they're slightly more secure, so that's what the session used.

The drum-roll moment: Bolt flagged "missing secrets — the following secrets are used in the code but don't exist: OPENAI_API_KEY" with a button straight to secret settings. Paste the key, create the secret, then run the feature: "launch a mission to meet Jeff Bezos by October" → research his schedule (assigned to the researcher), contact his team, coordinate travel (assigned to the driver), prepare a meeting agenda — each with a description, priority and due date. Approve, deny, then "add tasks to board". It worked.

---

10. Design, landing page and the mega prompt

The app looked boring, so: **superdesign.dev** → library of pre-built styles (Bold Retro Modernism, Bajas, Windows 98, Disruptor…). Pick one, copy the style, paste it into Bolt with *"please redesign the entire application with this style, making sure all existing functionality works properly."*

Then the **mega prompt** — three things in one, saved to the whiteboard for the group:

1. **Redesign** the whole application in the copied style, keeping all existing functionality.

2. **Standard authentication** — sign up and sign in with email and password, working with the database, keeping all existing functionality.
3. **A landing page** with sign-up/sign-in, and copy that is deliberately outrageous: written on a **problem–agitate–solution** framework, in the style of top copywriters (Sabri Suby, Alex Hormozi, David Ogilvy), painting the difference against competitors, addressing what keeps the prospect up at night — and a design that's stunning and deliberately *not* a standard symmetric layout.

The generated result is a dark, high-contrast, lime-accented page built around the pain of bloated project management software: the 3 AM cold sweat about whether anyone finished the API integration, forty-seven swim lanes, tickets in progress since March, ceremony about ceremony, the two-person startup in Lisbon shipping what you've had in planning since Q2 — and the promise: one board, an AI brain, zero theatre.

---

11. The six commandments of vibe coding

1. **Start with a plan.** AI can't read your mind, it can only read scripts. Create a clear PRD before coding — explore, plan, build. Do competitor research for a comprehensive feature set. This is what separates you from everyone else.
2. **Give clear, specific instructions.** Not "build a project management app" or "build a recipe app" — name the features and what they let you do.
3. **Build and test in small pieces.** Never pile features onto an unverified foundation.
4. **Save your work constantly.** Use version history / checkpoints in Bolt so you can rewind to the last working version with one click.
5. **Don't panic — don't be afraid to start fresh.** The hard part isn't the *how* any more; AI implements. The hard part is the *what*: what am I building, what problem does it solve, what are the core features, what does it look like. If the project stops cooperating or was built on the wrong foundation, restarting is cheap.
6. **Use platform guidelines.** Each tool has its own dialect — that's stage three of the PRD journey.

---

12. GitHub, free credits, and what Bolt is actually for

GitHub: a platform for storing projects in **repositories** ("repos" — a place to store files). The industry standard for storing code bases: secure, free, private. In Bolt there's a **Connect to GitHub** button → sign in → name it → create repository → it auto-syncs every change. Why bother: the code is then **yours forever**, safe even if the tool has a problem.

The free-credits hack: a new Bolt account gets a trial with monthly and daily credits. Export the code to GitHub, open a new account with fresh credits, import the code, keep building, repeat when you run out. Annoying and not graceful — but useful when there's no budget yet, and the plan is to reinvest once you have paying users.

Reality check — what Bolt.new is good for: rapid prototyping, MVPs (roughly 10–1,000 users), validating ideas, landing your first customers, learning and experimentation, making your first dollar online. **What it's not good for:** apps needing millions of users, complex enterprise systems, financial or medical compliance. For that: build it, get paying users, then bring in a senior developer — or move the code into **Claude Code** or **Codex**, which are more technically demanding (terminals, code files) but better for enterprise-level scaffolding.

Shipping it: publish to a live URL from inside Bolt. **Custom domain** — buy one in Bolt or add one you own, follow the instructions, live in ten to fifteen minutes. **Payments** — create a Stripe account, click Stripe, paste the API key, retrieve your products, and you can be selling inside the app in about ten minutes. If stuck, ask ChatGPT or Claude: *"I'm a non-technical builder, I built an application in Bolt.new, help me connect it to Stripe step by step."*

---

13. Closing

When things break: look at the problem, look at the error, don't panic. Tell AI what you need it to do and have it fix it — or roll back to a working version and try again.

The final line of the deck: *"You don't need to be technical. You just need to be specific. AI will handle the rest."*

Three attributes for success — whatever your definition of success is:
1. **Willingness to learn** — the moment you decide you don't want to learn is the moment you decide you don't want to grow.
2. **Willingness to work hard** — if success were easy, everyone would have it.
3. **Knowing you're not going to quit.**

The Japanese train story: commuters complained the trains took too long — so the railway put a timer at the top of the carriage showing the arrival time at the next station, and the complaints stopped. People don't mind things taking long; they mind not knowing when they'll arrive. So if you're willing to learn, willing to work hard, and you won't quit, it's not a matter of *if* — it's a matter of *when*.

Appendix — what's in `vibe-coding-visuals/`

183 time-stamped frames (HH-MM-SS) covering the whole session: the deck's own slides, every stage of the Excalidraw whiteboard (restaurant analogy architecture, ideation, EPB, Pareto, PRD journey, database and Spotify, CRUD, API key flow), and the live screens (Claude research and PRD, Bolt.new builds, database tables, secrets, OpenAI platform, superdesign, the generated landing page, the six commandments and GitHub slides).

Section	Time range	Examples
Deck slides	3:47–3:51, 4:15–4:16, 4:42–4:50, 5:40–5:41	`03-48-24` vibe coding title, `03-50-21` today's build, `03-51-45` restaurant analogy, `04-18-10` the PRD journey, `04-50-50` the build timeline, `05-40-06` six commandments
Whiteboard / diagrams	3:54–4:09, 4:29–4:34, 4:42–4:47, 5:12–5:26, 5:39–5:47	`03-55-44` VIBE CODING 101, `03-58-26` restaurant analogy, `04-03-06` two ways to ideate, `04-08-34` EPB framework, `04-31-26` Spotify relational database, `05-16-15` API key steps, `05-43-46` API key flow
Research & planning screens	4:10–4:28	`04-13-33` competitor feature harvest, `04-17-19` consolidated feature set, `04-22-37` TaskFlow AI PRD, `04-29-12` Bolt prompting best practices
Build & demo screens	4:46–5:10, 5:26–5:34	`04-49-53` Bolt project view, `05-05-17` task board, `05-30-10` AI task breakdown, `05-33-28` design styles
Landing page & ship	5:47–5:53	`05-48-30` generated landing page, `05-53-47` closing slide